

1. What is Instruction Set Architecture (ISA)?

Instruction Set Architecture (ISA) is the interface between **software** (programs) and **hardware** (processor).

It tells the programmer:

- What instructions are available (ADD, SUB, MUL, DIV, LOAD, STORE...)
- How operands are stored
- How many operands an instruction has
- Which registers exist
- Addressing modes
- Data sizes (8-bit, 16-bit, 32-bit, 64-bit)

In simple words,

ISA = The language understood by the CPU.

The lecture also mentions that ISAs often survive for many processor generations (for example, Intel x86).

2. ISA Design Issues

According to your slides, ISA design depends on four major things:

(1) Number of operands

Instructions may have

- 0 operands
- 1 operand
- 2 operands
- 3 operands

Example

ADD

0 operand (Stack)

ADD X

1 operand (Accumulator)

ADD R1,R2

2 operands

ADD R1,R2,R3

3 operands

(2) Location of operands

Operands may be stored in

- Stack
- Accumulator
- Memory
- Registers

(3) Addressing mode

Examples

- Register
- Immediate
- Indirect
- Relative

(4) Supported operations

Examples

ADD
SUB
MUL
DIV
AND
OR
MOVE
CMP
JMP

3. Evolution of Instruction Sets

Your lecture lists five architectures.

Architecture	Operand Location
Stack	Stack
Accumulator	Accumulator

Memory-Memory	Memory
Register-Memory	Register + Memory
Register-Register	Registers

4. Stack Architecture

Idea

Everything happens on a **stack**.

The programmer never specifies registers.

Operations always use the **top two stack values**.

Common Instructions

PUSH X
POP X
ADD
SUB
MUL
DIV

Example

$Z = X + Y$

Solution

PUSH X
PUSH Y
ADD
POP Z

Exactly as shown in your notes.

How ADD works

Before

Y
X

After

$X+Y$

Top two numbers disappear.

Their sum is pushed back.

Advantages

- Simple instructions
- No register names

Disadvantages

- Too many PUSH and POP instructions
- Harder for the compiler

5. Accumulator Architecture

Idea

There is only **one special register** called the **Accumulator (ACC)**.

Every arithmetic operation automatically uses ACC.

Example

LOAD X

means

$ACC = X$

ADD Y

means

$ACC = ACC + Y$

STORE Z

means

$Z = ACC$

Example

$$Z = X + Y$$

Program

```
LOAD X  
ADD Y  
STORE Z
```

Exactly matches your lecture.

Advantages

- Few registers
- Simple hardware

Disadvantages

- Too many LOAD and STORE instructions

6. Memory-Memory Architecture

Idea

Everything stays in **memory**.

Arithmetic instructions directly use memory locations.

Registers are unnecessary.

Example

```
ADD C, A, B
```

means

$$C = A + B$$

All operands are memory locations.

There are two versions.

Three operand

ADD X,Y,Z

means

$X = Y + Z$

Two operand

MOV D,A

ADD D,B

means

D=A

D=D+B

Your slides show both versions.

Advantages

- Very easy to write programs

Disadvantages

- Slow because every instruction accesses memory

7. Register-Memory Architecture

Idea

One operand comes from a **register**.

The other operand comes from **memory**.

Example

LOAD R2,X

ADD R2,Y

STORE Z,R2

means

$R2 = X$

$R2 = R2 + Y$

$Z = R2$

Exactly from your notes.

Advantages

- Faster than Memory-Memory
- Less memory traffic

Disadvantages

- Instructions become more complicated

8. Register-Register Architecture (Load-Store)

Also called

Load-Store Architecture

Only

LOAD

STORE

can access memory.

Arithmetic uses registers only.

Example

LOAD R1,X

LOAD R2,Y

ADD R3,R1,R2

STORE Z,R3

means

$R1 = X$

$R2 = Y$

$R3 = R1 + R2$

$Z = R3$

Exactly as in the lecture.

Advantages

- Fastest architecture
- Easy pipelining
- Used in RISC processors

Disadvantages

- Needs many registers

Architecture Comparison

Architecture	Operands	Arithmetic Uses
Stack	0	Stack
Accumulator	1	ACC
Memory-Memory	2/3	Memory
Register-Memory	2	Register + Memory
Register-Register	3	Registers

How to Solve Mathematical Expressions

This is the most common exam question.

Question 1

$$Z = A + B$$

Stack

```
PUSH A
PUSH B
ADD
POP Z
```


Accumulator

```
LOAD A
ADD B
STORE Z
```

Memory-Memory (3 operand)

```
ADD Z,A,B
```

Register-Memory

```
LOAD R1,A
ADD R1,B
STORE Z,R1
```

Register-Register

```
LOAD R1,A
LOAD R2,B
ADD R3,R1,R2
STORE Z,R3
```

Question 2

$Z = (A+B) * C$

Stack

```
PUSH A
PUSH B
ADD
PUSH C
MUL
POP Z
```

Accumulator

```
LOAD A
ADD B
STORE D
LOAD D
```

MUL C
STORE Z

Memory-Memory

ADD D,A,B
MUL Z,D,C

Register-Memory

LOAD R1,A
ADD R1,B
MUL R1,C
STORE Z,R1

Register-Register

LOAD R1,A
LOAD R2,B
LOAD R3,C
ADD R4,R1,R2
MUL R5,R4,R3
STORE Z,R5

Question 3

$Z=A-B+C$

Stack

PUSH A
PUSH B
SUB
PUSH C
ADD
POP Z

Accumulator

LOAD A
SUB B

ADD C
STORE Z

Memory-Memory

SUB D,A,B
ADD Z,D,C

Register-Memory

LOAD R1,A
SUB R1,B
ADD R1,C
STORE Z,R1

Register-Register

LOAD R1,A
LOAD R2,B
LOAD R3,C
SUB R4,R1,R2
ADD R5,R4,R3
STORE Z,R5

Question 4 (Same as Your Lecture)

$$Y = A/B - (A - C \times B)$$

The lecture provides complete solutions for each architecture.

Exam Practice Questions

Try writing these using **all five architectures**:

1. $Z = A + B$
2. $Z = A - B$
3. $Z = A \times B$
4. $Z = A / B$
5. $Z = (A + B) \times C$
6. $Z = A + B \times C$

7. $Z = (A - B) + C$
8. $Z = (A + B) / C$
9. $Z = A \times B + C$
10. $Z = A / B - C$
11. $Z = (A + B) \times (C - D)$
12. $Z = (A \times B) + (C \times D)$
13. $Z = A + B + C + D$
14. $Z = A \times (B + C)$
15. $Y = A / B - (A - C \times B)$ (from your lecture)

Quick Memory Tricks

- **Stack** → Think **PUSH / POP**.
- **Accumulator** → Think **one special ACC register**.
- **Memory-Memory** → Arithmetic happens directly on **memory locations**.
- **Register-Memory** → One operand in a **register**, one in **memory**.
- **Register-Register (Load-Store)** → **LOAD** data into registers, perform all arithmetic on registers, then **STORE** the result back to memory.

These patterns are exactly what exam questions typically test when asking you to convert mathematical expressions into instruction sequences.

Sun, Jun 21 at 5:53 PM

$Y = A/B - (A - C \times B)$

can u give answers of all types of architectures

Yes. Below are the answers for the **15 practice questions** using **all five architectures**. I use a temporary variable (D, E) or temporary register (R4, R5) whenever an intermediate result is needed, just like your lecture notes.

1. $Z = A + B$

Stack

```
PUSH A
PUSH B
ADD
POP Z
```

Accumulator

```
LOAD A
ADD B
STORE Z
```

Memory-Memory

```
ADD Z,A,B
```

Register-Memory

```
LOAD R1,A
ADD R1,B
STORE Z,R1
```

Register-Register

```
LOAD R1,A
LOAD R2,B
ADD R3,R1,R2
STORE Z,R3
```

2. $Z = A - B$

Stack

```
PUSH A
PUSH B
SUB
POP Z
```

Accumulator

```
LOAD A
SUB B
STORE Z
```

Memory-Memory

```
SUB Z,A,B
```

Register-Memory

```
LOAD R1,A  
SUB R1,B  
STORE Z,R1
```

Register-Register

```
LOAD R1,A  
LOAD R2,B  
SUB R3,R1,R2  
STORE Z,R3
```

3. $Z = A \times B$

Stack

```
PUSH A  
PUSH B  
MUL  
POP Z
```

Accumulator

```
LOAD A  
MUL B  
STORE Z
```

Memory-Memory

```
MUL Z,A,B
```

Register-Memory

```
LOAD R1,A  
MUL R1,B  
STORE Z,R1
```

Register-Register

```
LOAD R1,A  
LOAD R2,B
```

```
MUL R3,R1,R2
STORE Z,R3
```

4. $Z = A \div B$

Stack

```
PUSH A
PUSH B
DIV
POP Z
```

Accumulator

```
LOAD A
DIV B
STORE Z
```

Memory-Memory

```
DIV Z,A,B
```

Register-Memory

```
LOAD R1,A
DIV R1,B
STORE Z,R1
```

Register-Register

```
LOAD R1,A
LOAD R2,B
DIV R3,R1,R2
STORE Z,R3
```

5. $Z = (A + B) \times C$

Stack

```
PUSH A
PUSH B
```

ADD
PUSH C
MUL
POP Z

Accumulator

LOAD A
ADD B
STORE D
LOAD D
MUL C
STORE Z

Memory-Memory

ADD D,A,B
MUL Z,D,C

Register-Memory

LOAD R1,A
ADD R1,B
MUL R1,C
STORE Z,R1

Register-Register

LOAD R1,A
LOAD R2,B
LOAD R3,C
ADD R4,R1,R2
MUL R5,R4,R3
STORE Z,R5

6. $Z = A + (B \times C)$

Stack

PUSH A
PUSH B
PUSH C
MUL

ADD
POP Z

Accumulator

LOAD B
MUL C
STORE D
LOAD A
ADD D
STORE Z

Memory-Memory

MUL D,B,C
ADD Z,A,D

Register-Memory

LOAD R1,B
MUL R1,C
ADD R1,A
STORE Z,R1

Register-Register

LOAD R1,A
LOAD R2,B
LOAD R3,C
MUL R4,R2,R3
ADD R5,R1,R4
STORE Z,R5

$$7. Z = (A - B) + C$$

Stack

PUSH A
PUSH B
SUB
PUSH C
ADD
POP Z

Accumulator

```
LOAD A
SUB B
ADD C
STORE Z
```

Memory-Memory

```
SUB D,A,B
ADD Z,D,C
```

Register-Memory

```
LOAD R1,A
SUB R1,B
ADD R1,C
STORE Z,R1
```

Register-Register

```
LOAD R1,A
LOAD R2,B
LOAD R3,C
SUB R4,R1,R2
ADD R5,R4,R3
STORE Z,R5
```

8. $Z = (A + B) \div C$

Stack

```
PUSH A
PUSH B
ADD
PUSH C
DIV
POP Z
```

Accumulator

```
LOAD A
ADD B
```

STORE D
LOAD D
DIV C
STORE Z

Memory-Memory

ADD D,A,B
DIV Z,D,C

Register-Memory

LOAD R1,A
ADD R1,B
DIV R1,C
STORE Z,R1

Register-Register

LOAD R1,A
LOAD R2,B
LOAD R3,C
ADD R4,R1,R2
DIV R5,R4,R3
STORE Z,R5

9. $Z = (A \times B) + C$

Stack

PUSH A
PUSH B
MUL
PUSH C
ADD
POP Z

Accumulator

LOAD A
MUL B
ADD C
STORE Z

Memory-Memory

```
MUL D,A,B  
ADD Z,D,C
```

Register-Memory

```
LOAD R1,A  
MUL R1,B  
ADD R1,C  
STORE Z,R1
```

Register-Register

```
LOAD R1,A  
LOAD R2,B  
LOAD R3,C  
MUL R4,R1,R2  
ADD R5,R4,R3  
STORE Z,R5
```

$$10. Z = (A \div B) - C$$

Stack

```
PUSH A  
PUSH B  
DIV  
PUSH C  
SUB  
POP Z
```

Accumulator

```
LOAD A  
DIV B  
SUB C  
STORE Z
```

Memory-Memory

```
DIV D,A,B  
SUB Z,D,C
```

Register-Memory

```
LOAD R1,A
DIV R1,B
SUB R1,C
STORE Z,R1
```

Register-Register

```
LOAD R1,A
LOAD R2,B
LOAD R3,C
DIV R4,R1,R2
SUB R5,R4,R3
STORE Z,R5
```

$$11. Z = (A + B) \times (C - D)$$

Stack

```
PUSH A
PUSH B
ADD
PUSH C
PUSH D
SUB
MUL
POP Z
```

Accumulator

```
LOAD A
ADD B
STORE E
LOAD C
SUB D
MUL E
STORE Z
```

Memory-Memory

```
ADD E,A,B
SUB D1,C,D
MUL Z,E,D1
```

Register-Memory

```
LOAD R1,A
ADD R1,B
LOAD R2,C
SUB R2,D
MUL R1,R2
STORE Z,R1
```

Register-Register

```
LOAD R1,A
LOAD R2,B
LOAD R3,C
LOAD R4,D
ADD R5,R1,R2
SUB R6,R3,R4
MUL R7,R5,R6
STORE Z,R7
```

$$12. Z = (A \times B) + (C \times D)$$

Stack

```
PUSH A
PUSH B
MUL
PUSH C
PUSH D
MUL
ADD
POP Z
```

Accumulator

```
LOAD A
MUL B
```

```
STORE E
LOAD C
MUL D
ADD E
STORE Z
```

Memory-Memory

```
MUL E,A,B
MUL F,C,D
ADD Z,E,F
```

Register-Memory

```
LOAD R1,A
MUL R1,B
LOAD R2,C
MUL R2,D
ADD R1,R2
STORE Z,R1
```

Register-Register

```
LOAD R1,A
LOAD R2,B
LOAD R3,C
LOAD R4,D
MUL R5,R1,R2
MUL R6,R3,R4
ADD R7,R5,R6
STORE Z,R7
```

13. $Z = A + B + C + D$

Stack

```
PUSH A
PUSH B
ADD
PUSH C
ADD
PUSH D
ADD
POP Z
```

Accumulator

```
LOAD A
ADD B
ADD C
ADD D
STORE Z
```

Memory-Memory

```
ADD E,A,B
ADD F,E,C
ADD Z,F,D
```

Register-Memory

```
LOAD R1,A
ADD R1,B
ADD R1,C
ADD R1,D
STORE Z,R1
```

Register-Register

```
LOAD R1,A
LOAD R2,B
LOAD R3,C
LOAD R4,D
ADD R5,R1,R2
ADD R6,R5,R3
ADD R7,R6,R4
STORE Z,R7
```

14. $Z = A \times (B + C)$

Stack

```
PUSH A
PUSH B
PUSH C
ADD
MUL
POP Z
```


Accumulator

```
LOAD B
ADD C
STORE D
LOAD A
MUL D
STORE Z
```

Memory-Memory

```
ADD D,B,C
MUL Z,A,D
```

Register-Memory

```
LOAD R1,B
ADD R1,C
MUL R1,A
STORE Z,R1
```

Register-Register

```
LOAD R1,A
LOAD R2,B
LOAD R3,C
ADD R4,R2,R3
MUL R5,R1,R4
STORE Z,R5
```

15. $Y = A/B - (A - C \times B)$

These are the **same answers as your lecture notes**.

Stack

```
PUSH A
PUSH B
DIV
PUSH A
PUSH C
PUSH B
MUL
```

SUB
SUB
POP Y

Accumulator

LOAD C
MUL B
STORE D
LOAD A
SUB D
STORE D
LOAD A
DIV B
SUB D
STORE Y

Memory-Memory

DIV D,A,B
MUL E,C,B
SUB E,A,E
SUB Y,D,E

Register-Memory

LOAD R1,A
DIV R1,B
STORE D,R1
LOAD R2,C
MUL R2,B
LOAD R3,A
SUB R3,R2
LOAD R1,D
SUB R1,R3
STORE Y,R1

Register-Register (Load-Store)

LOAD R1,A
LOAD R2,B
LOAD R3,C
DIV R4,R1,R2
MUL R5,R3,R2
SUB R5,R1,R5

```
SUB R4,R4,R5  
STORE Y,R4
```